Lab: SVFIR and Control-Flow Reachability

(Week 2)

Yulei Sui

School of Computer Science and Engineering

University of New South Wales, Australia

Quiz-1 + Lab-Exercise-1 + Assignment-1

- A set of quizzes on WebCMS (5 points) due on **Week 3 Wednesday 23:59**
  - LLVM compiler and its intermediate representation
  - Code graphs (including ICFG and PAG)
- Lab-Exercise-1 (5 points) due on **Week 3 Wednesday 23:59**
  - Implement a graph traversal on a general graph
- Assignment-1 (20 points) due on **Week 4 Wednesday 23:59**
  - **Control-flow**: Implement a context-sensitive graph traversal on a CodeGraph (i.e., ICFG) and collect **feasible** paths from a source to a sink node on the ICFG.
  - **Data-flow**: Implement field-sensitive Andersen's inclusion-based constraint solving for points-to analysis
  - Implement a taint checker **using control-flow and data-flow analysis**.

Quiz-1 + Lab-Exercise-1 + Assignment-1

- A set of quizzes on WebCMS (5 points) due on **Week 3 Wednesday 23:59**
  - LLVM compiler and its intermediate representation
  - Code graphs (including ICFG and PAG)
- Lab-Exercise-1 (5 points) due on **Week 3 Wednesday 23:59**
  - Implement a graph traversal on a general graph
- Assignment-1 (20 points) due on **Week 4 Wednesday 23:59**
  - **Control-flow**: Implement a context-sensitive graph traversal on a CodeGraph (i.e., ICFG) and collect **feasible** paths from a source to a sink node on the ICFG.
  - **Data-flow**: Implement field-sensitive Andersen's inclusion-based constraint solving for points-to analysis
  - Implement a taint checker **using control-flow and data-flow analysis**.
  - **Specification and code template**: `https://github.com/SVF-tools/Software-Security-Analysis/wiki/Assignment-1`
  - **SVF APIs for control- and data-flow analysis** `https://github.com/SVF-tools/Software-Security-Analysis/wiki/SVF-API`

Understanding LLVM-IR and SVF-IR

- (1) Compile C programs under `SVFIR/src` into their LLVM IR and print their SVF IR (PAG, ICFG, Constraint Graph).
  - `https://github.com/SVF-tools/Software-Security-Analysis/wiki/SVFIR`
  - Understand the mapping from a C program to its corresponding LLVM IR
  - Understand the mapping from LLVM IR to its corresponding SVF IR
- (2) Generate and visualize the graph representation of SVF IR (e.g., `example.ll.pag.dot`, `example.ll.icfg.dot`, `consG.ll.dot`).
  - `https://github.com/SVF-tools/Software-Security-Analysis/wiki/SVFIR#4-visualize-icfg-constraint-graph-and-svfirpag-graph`
- (3) Write code to iterate SVFVars and print nodes/edges of PAG and ICFG.
  - `https://github.com/SVF-tools/Software-Security-Analysis/blob/main/SVFIR/SVFIR.cpp#L74-L98` (C++)
  - `https://github.com/SVF-tools/SVF-Python/tree/main/demo` (Python)
- (4) More about LLVM IR and SVF's graph representation
  - LLVM language manual `https://llvm.org/docs/LangRef.html`
  - SVF website `https://github.com/SVF-tools/SVF`

Control-Flow Reachability

(Week 2)

Yulei Sui, Mohamad Barbar, Hanyuan Li, Angela He

School of Computer Science and Engineering

University of New South Wales, Australia

Context-Sensitive Control-Flow Reachability (Algorithm)

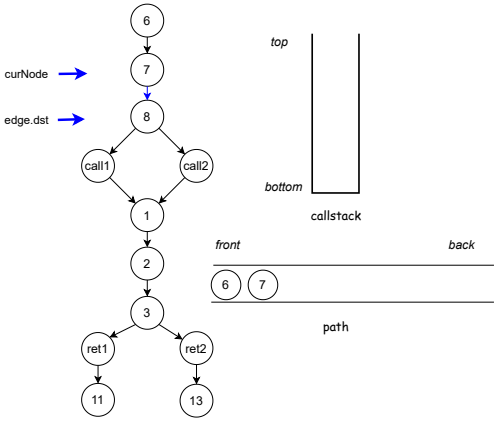
Algorithm 1: 1 Context sensitive control-flow reachability

Input : curNode : ICFGNode snk : ICFGNode path : vector⟨ICFGNode⟩
callstack : vector⟨SVFInstruction⟩ visited : set⟨ICFGNode, callstack⟩;

```
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4        return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8        collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11         dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13         callstack.push_back(edge.getCallSite());
14         dfs(edge.dst, snk);
15         callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17         if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18             callstack.pop_back();
19             dfs(edge.dst, snk);
20             callstack.push_back(edge.getCallSite());
21         else if callstack == ∅ then
22             dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



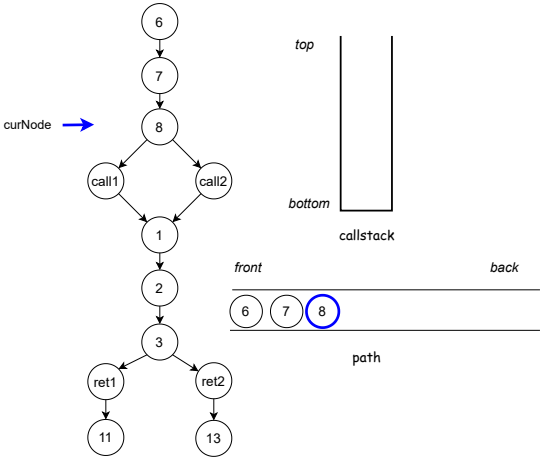
Algorithm 2: 1 Context sensitive control-flow reachability

Input : curNode : ICFGNode snk : ICFGNode path : vector⟨ICFGNode⟩
callstack : vector⟨SVFInstruction⟩ visited : set⟨ICFGNode, callstack⟩;

```
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4      return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8      collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11       dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13       callstack.push_back(edge.getCallSite());
14       dfs(edge.dst, snk);
15       callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17       if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18         callstack.pop_back();
19         dfs(edge.dst, snk);
20         callstack.push_back(edge.getCallSite());
21       else if callstack == ∅ then
22         dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG

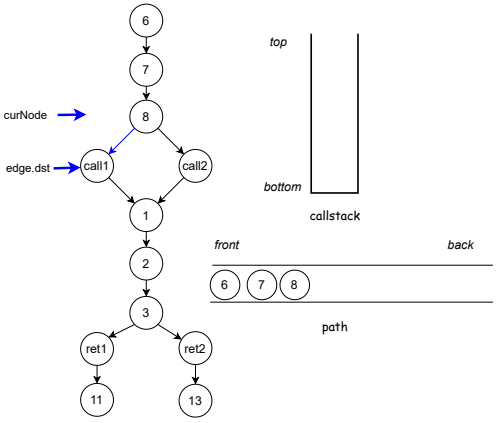


Algorithm 3: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode   snk : ICFGNode   path : vector⟨ICFGNode⟩
        callstack : vector⟨SVFInstruction⟩   visited : set⟨ICFGNode, callstack⟩;
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4        return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8        collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10       if edge.isIntraCFGEdge() then
11           dfs(edge.dst, snk);
12       else if edge.isCallCFGEdge() then
13           callstack.push_back(edge.getCallSite());
14           dfs(edge.dst, snk);
15           callstack.pop_back();
16       else if edge.isRetCFGEdge() then
17           if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18               callstack.pop_back();
19               dfs(edge.dst, snk);
20               callstack.push_back(edge.getCallSite());
21           else if callstack == ∅ then
22               dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



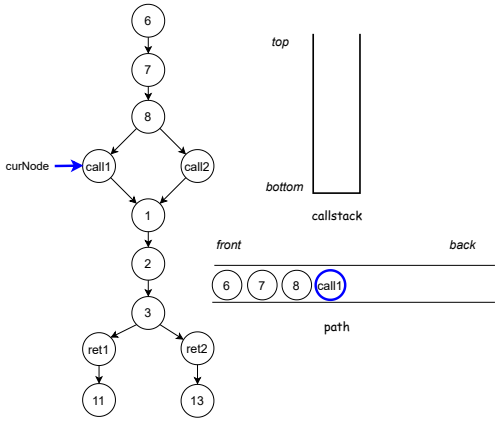
Algorithm 4: 1 Context sensitive control-flow reachability

Input : curNode : ICFGNode snk : ICFGNode path : vector⟨ICFGNode⟩
callstack : vector⟨SVFInstruction⟩ visited : set⟨ICFGNode, callstack⟩;

```
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4      return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8      collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11       dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13       callstack.push_back(edge.getCallSite());
14       dfs(edge.dst, snk);
15       callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17       if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18         callstack.pop_back();
19         dfs(edge.dst, snk);
20         callstack.push_back(edge.getCallSite());
21       else if callstack == ∅ then
22         dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



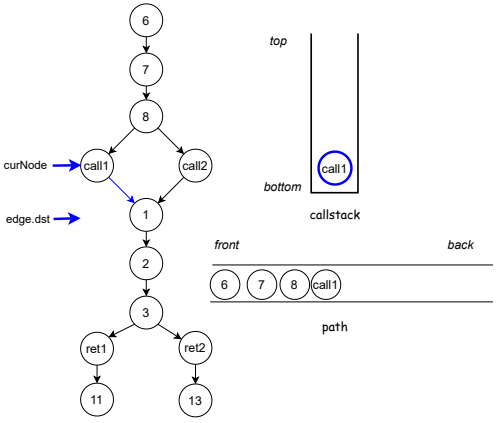
Algorithm 5: 1 Context sensitive control-flow reachability

Input : curNode : ICFGNode snk : ICFGNode path : vector⟨ICFGNode⟩
callstack : vector⟨SVFInstruction⟩ visited : set⟨ICFGNode, callstack⟩;

```
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4      return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8      collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11       dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13       callstack.push_back(edge.getCallSite());
14       dfs(edge.dst, snk);
15       callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17       if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18         callstack.pop_back();
19         dfs(edge.dst, snk);
20         callstack.push_back(edge.getCallSite());
21       else if callstack == ∅ then
22         dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



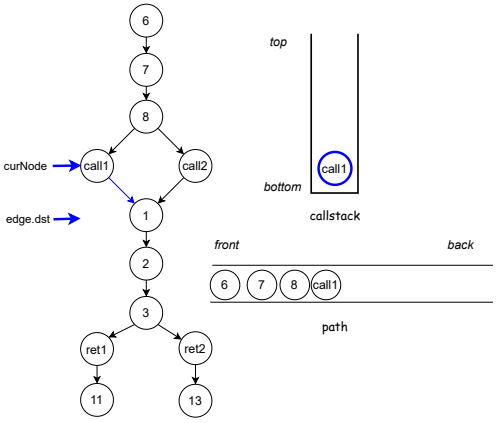
Algorithm 6: 1 Context sensitive control-flow reachability

Input : curNode : ICFGNode snk : ICFGNode path : vector⟨ICFGNode⟩
callstack : vector⟨SVFInstruction⟩ visited : set⟨ICFGNode, callstack⟩;

```
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4      return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8      collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11       dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13       callstack.push_back(edge.getCallSite());
14       dfs(edge.dst, snk);
15       callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17       if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18         callstack.pop_back();
19         dfs(edge.dst, snk);
20         callstack.push_back(edge.getCallSite());
21       else if callstack == ∅ then
22         dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



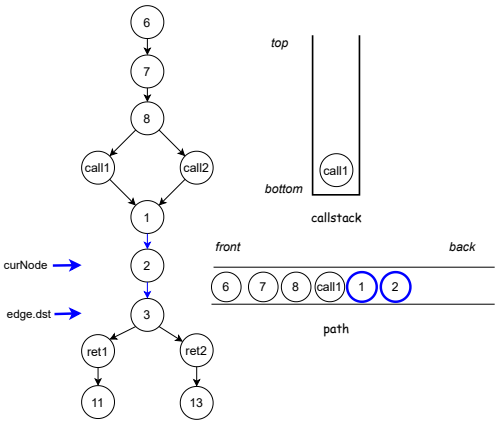
Algorithm 7: 1 Context sensitive control-flow reachability

Input : curNode : ICFGNode snk : ICFGNode path : vector⟨ICFGNode⟩
callstack : vector⟨SVFInstruction⟩ visited : set⟨ICFGNode, callstack⟩;

```
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4      return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8      collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11       dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13       callstack.push_back(edge.getCallSite());
14       dfs(edge.dst, snk);
15       callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17       if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18         callstack.pop_back();
19         dfs(edge.dst, snk);
20         callstack.push_back(edge.getCallSite());
21       else if callstack == ∅ then
22         dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



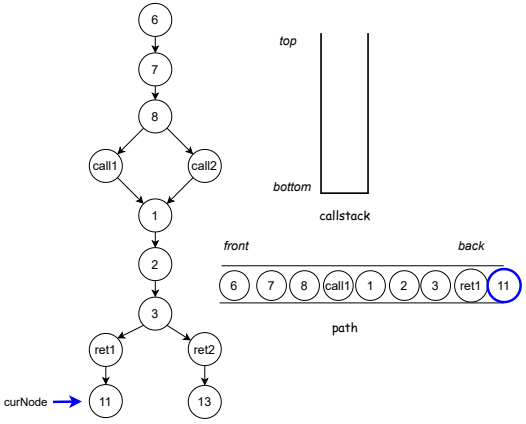
Algorithm 8: 1 Context sensitive control-flow reachability

Input : curNode : ICFGNode snk : ICFGNode path : vector⟨ICFGNode⟩
callstack : vector⟨SVFInstruction⟩ visited : set⟨ICFGNode, callstack⟩;

```
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4      return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8      collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11       dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13       callstack.push_back(edge.getCallSite());
14       dfs(edge.dst, snk);
15       callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17       if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18         callstack.pop_back();
19         dfs(edge.dst, snk);
20         callstack.push_back(edge.getCallSite());
21       else if callstack == ∅ then
22         dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG

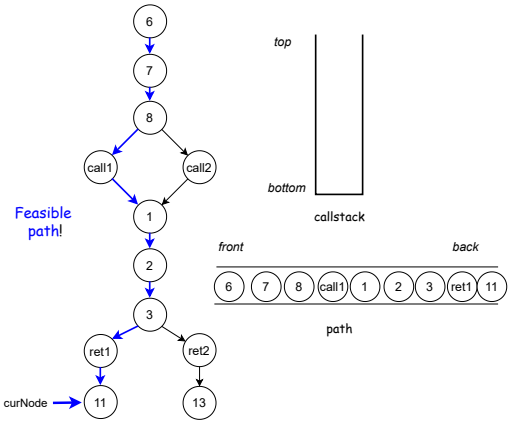


Algorithm 9: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode   snk : ICFGNode   path : vector⟨ICFGNode⟩
        callstack : vector⟨SVFInstruction⟩   visited : set⟨ICFGNode, callstack⟩;
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4      return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8      collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11       dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13       callstack.push_back(edge.getCallSite());
14       dfs(edge.dst, snk);
15       callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17       if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18         callstack.pop_back();
19         dfs(edge.dst, snk);
20         callstack.push_back(edge.getCallSite());
21       else if callstack == ∅ then
22         dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



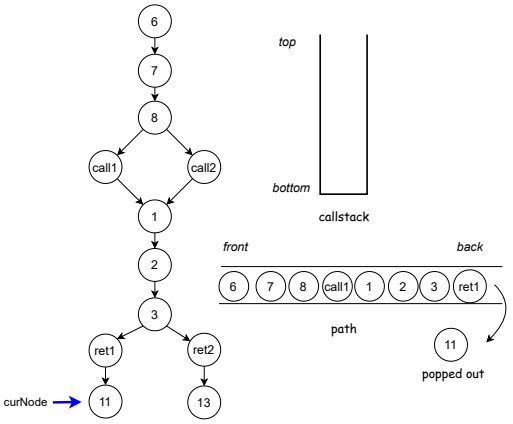
Algorithm 10: 1 Context sensitive control-flow reachability

Input : curNode : ICFGNode snk : ICFGNode path : vector⟨ICFGNode⟩
callstack : vector⟨SVFInstruction⟩ visited : set⟨ICFGNode, callstack⟩;

```
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4      return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8      collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11       dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13       callstack.push_back(edge.getCallSite());
14       dfs(edge.dst, snk);
15       callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17       if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18         callstack.pop_back();
19         dfs(edge.dst, snk);
20         callstack.push_back(edge.getCallSite());
21       else if callstack == ∅ then
22         dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



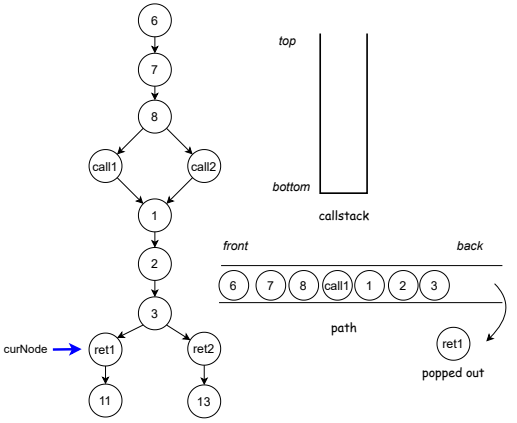
Algorithm 11: 1 Context sensitive control-flow reachability

Input : `curNode : ICFGNode  snk : ICFGNode  path : vector⟨ICFGNode⟩`
`callstack : vector⟨SVFInstruction⟩  visited : set⟨ICFGNode, callstack⟩;`

```
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4      return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8      collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11       dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13       callstack.push_back(edge.getCallSite());
14       dfs(edge.dst, snk);
15       callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17       if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18         callstack.pop_back();
19         dfs(edge.dst, snk);
20         callstack.push_back(edge.getCallSite());
21       else if callstack == ∅ then
22         dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



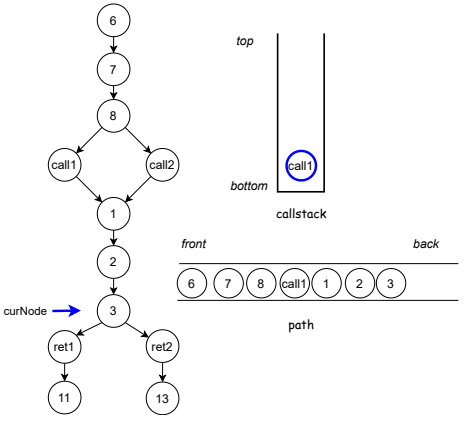
Algorithm 12: 1 Context sensitive control-flow reachability

Input : curNode : ICFGNode snk : ICFGNode path : vector⟨ICFGNode⟩
callstack : vector⟨SVFInstruction⟩ visited : set⟨ICFGNode, callstack⟩;

```
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4      return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8      collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11       dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13       callstack.push_back(edge.getCallSite());
14       dfs(edge.dst, snk);
15       callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17       if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18         callstack.pop_back();
19         dfs(edge.dst, snk);
20         callstack.push_back(edge.getCallSite());
21       else if callstack == ∅ then
22         dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



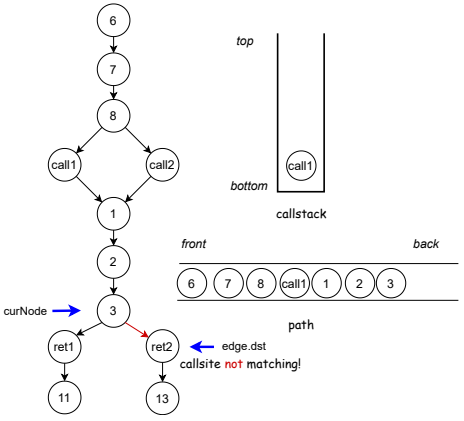
Algorithm 13: 1 Context sensitive control-flow reachability

Input : curNode : ICFGNode snk : ICFGNode path : vector⟨ICFGNode⟩
callstack : vector⟨SVFInstruction⟩ visited : set⟨ICFGNode, callstack⟩;

```
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4      return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8      collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11       dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13       callstack.push_back(edge.getCallSite());
14       dfs(edge.dst, snk);
15       callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17       if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18         callstack.pop_back();
19         dfs(edge.dst, snk);
20         callstack.push_back(edge.getCallSite());
21       else if callstack == ∅ then
22         dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



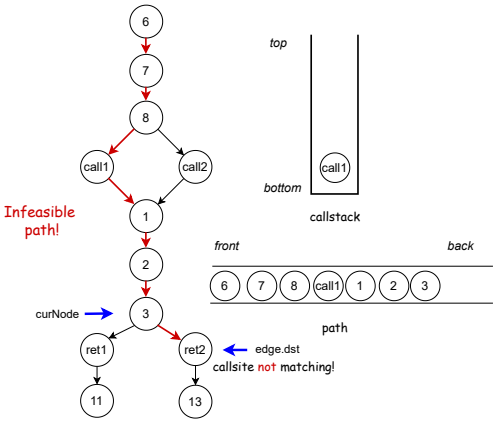
Algorithm 14: 1 Context sensitive control-flow reachability

Input : curNode : ICFGNode snk : ICFGNode path : vector⟨ICFGNode⟩
callstack : vector⟨SVFInstruction⟩ visited : set⟨ICFGNode, callstack⟩;

```
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4      return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8      collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11       dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13       callstack.push_back(edge.getCallSite());
14       dfs(edge.dst, snk);
15       callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17       if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18         callstack.pop_back();
19         dfs(edge.dst, snk);
20         callstack.push_back(edge.getCallSite());
21       else if callstack == ∅ then
22         dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



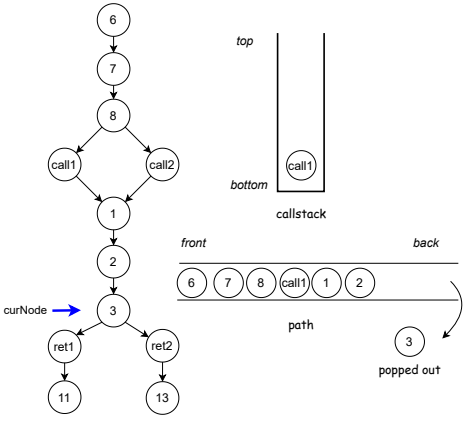
Algorithm 15: 1 Context sensitive control-flow reachability

Input : curNode : ICFGNode snk : ICFGNode path : vector⟨ICFGNode⟩
callstack : vector⟨SVFInstruction⟩ visited : set⟨ICFGNode, callstack⟩;

```
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4      return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8      collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11       dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13       callstack.push_back(edge.getCallSite());
14       dfs(edge.dst, snk);
15       callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17       if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18         callstack.pop_back();
19         dfs(edge.dst, snk);
20         callstack.push_back(edge.getCallSite());
21       else if callstack == ∅ then
22         dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG

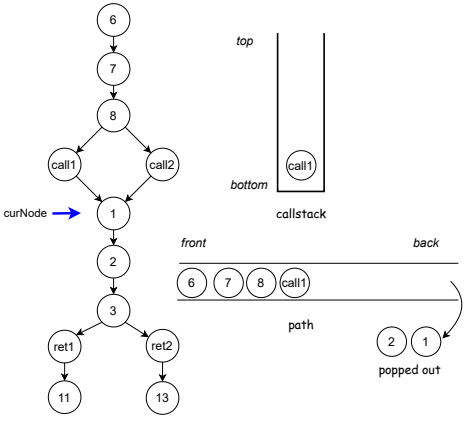


Algorithm 16: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector⟨ICFGNode⟩
        callstack : vector⟨SVFInstruction⟩  visited : set⟨ICFGNode, callstack⟩;
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4      return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8      collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11       dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13       callstack.push_back(edge.getCallSite());
14       dfs(edge.dst, snk);
15       callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17       if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18         callstack.pop_back();
19         dfs(edge.dst, snk);
20         callstack.push_back(edge.getCallSite());
21       else if callstack == ∅ then
22         dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



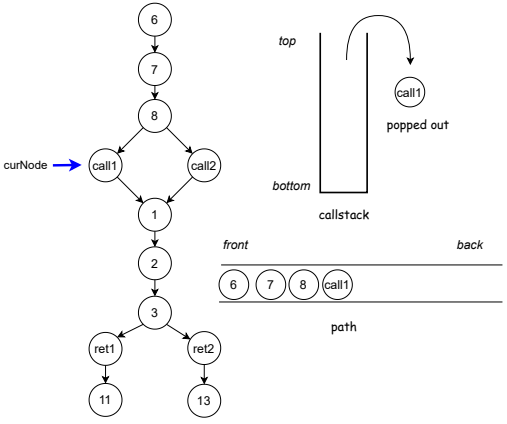
Algorithm 17: 1 Context sensitive control-flow reachability

Input : curNode : ICFGNode snk : ICFGNode path : vector⟨ICFGNode⟩
callstack : vector⟨SVFInstruction⟩ visited : set⟨ICFGNode, callstack⟩;

```
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4      return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8      collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11       dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13       callstack.push_back(edge.getCallSite());
14       dfs(edge.dst, snk);
15       callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17       if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18         callstack.pop_back();
19         dfs(edge.dst, snk);
20         callstack.push_back(edge.getCallSite());
21       else if callstack == ∅ then
22         dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



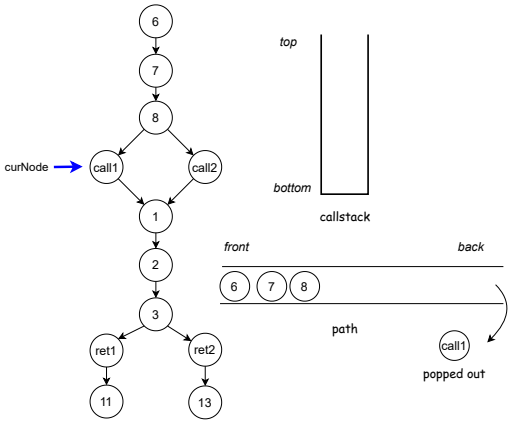
Algorithm 18: 1 Context sensitive control-flow reachability

Input : curNode : ICFGNode snk : ICFGNode path : vector⟨ICFGNode⟩
callstack : vector⟨SVFInstruction⟩ visited : set⟨ICFGNode, callstack⟩;

```
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4      return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8      collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11       dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13       callstack.push_back(edge.getCallSite());
14       dfs(edge.dst, snk);
15       callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17       if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18         callstack.pop_back();
19         dfs(edge.dst, snk);
20         callstack.push_back(edge.getCallSite());
21       else if callstack == ∅ then
22         dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG

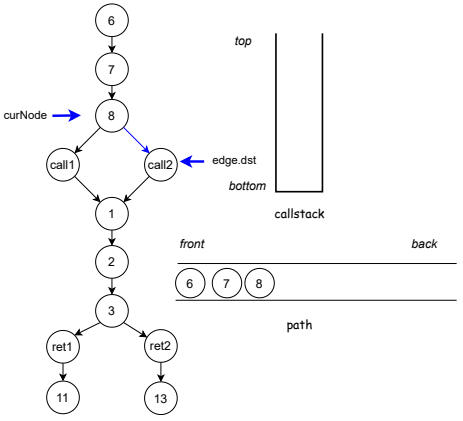


Algorithm 19: 1 Context sensitive control-flow reachability

```
Input : curNode : ICFGNode  snk : ICFGNode  path : vector⟨ICFGNode⟩
        callstack : vector⟨SVFInstruction⟩  visited : set⟨ICFGNode, callstack⟩;
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4      return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8      collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11       dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13       callstack.push_back(edge.getCallSite());
14       dfs(edge.dst, snk);
15       callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17       if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18         callstack.pop_back();
19         dfs(edge.dst, snk);
20         callstack.push_back(edge.getCallSite());
21       else if callstack == ∅ then
22         dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



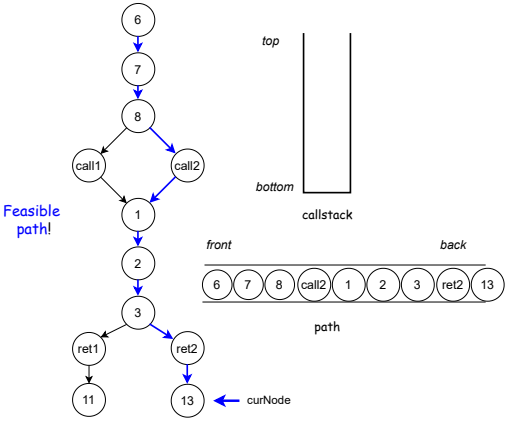
Algorithm 20: 1 Context sensitive control-flow reachability

Input : curNode : ICFGNode snk : ICFGNode path : vector⟨ICFGNode⟩
callstack : vector⟨SVFInstruction⟩ visited : set⟨ICFGNode, callstack⟩;

```
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4      return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8      collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11       dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13       callstack.push_back(edge.getCallSite());
14       dfs(edge.dst, snk);
15       callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17       if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18         callstack.pop_back();
19         dfs(edge.dst, snk);
20         callstack.push_back(edge.getCallSite());
21       else if callstack == ∅ then
22         dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG



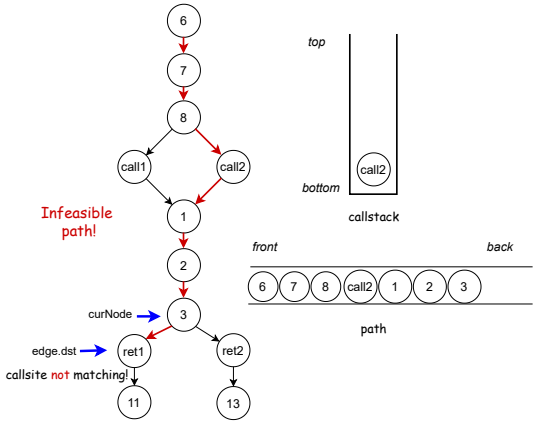
Algorithm 21: 1 Context sensitive control-flow reachability

input : curNode : ICFGNode snk : ICFGNode path : vector⟨ICFGNode⟩
callstack : vector⟨SVFInstruction⟩ visited : set⟨ICFGNode, callstack⟩;

```
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4      return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8      collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11       dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13       callstack.push_back(edge.getCallSite());
14       dfs(edge.dst, snk);
15       callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17       if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18         callstack.pop_back();
19         dfs(edge.dst, snk);
20         callstack.push_back(edge.getCallSite());
21       else if callstack == ∅ then
22         dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

Context-Sensitive Control-Flow Reachability

A feasible path from node 6 to node 11 on ICFG

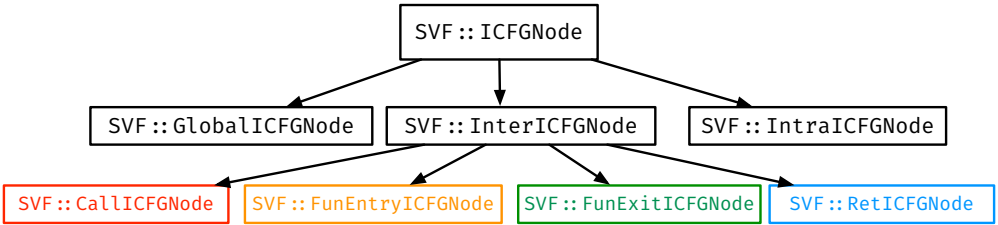


Algorithm 22: 1 Context sensitive control-flow reachability

Input : curNode : ICFGNode snk : ICFGNode path : vector⟨ICFGNode⟩
callstack : vector⟨SVFInstruction⟩ visited : set⟨ICFGNode, callstack⟩;

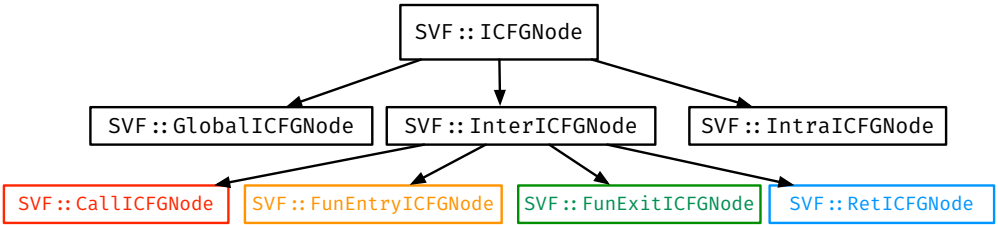
```
1  dfs(curNode, snk)
2    pair = ⟨curNode, callstack⟩;
3    if pair ∈ visited then
4      return;
5    visited.insert(pair);
6    path.push_back(curNode);
7    if src == snk then
8      collectICFGPath(path);
9    foreach edge ∈ curNode.getOutEdges() do
10     if edge.isIntraCFGEdge() then
11       dfs(edge.dst, snk);
12     else if edge.isCallCFGEdge() then
13       callstack.push_back(edge.getCallSite());
14       dfs(edge.dst, snk);
15       callstack.pop_back();
16     else if edge.isRetCFGEdge() then
17       if callstack ≠ ∅ && callstack.back() == edge.getCallSite() then
18         callstack.pop_back();
19         dfs(edge.dst, snk);
20         callstack.push_back(edge.getCallSite());
21       else if callstack == ∅ then
22         dfs(edge.dst, snk);
23   visited.erase(pair);
24   path.pop_back();
```

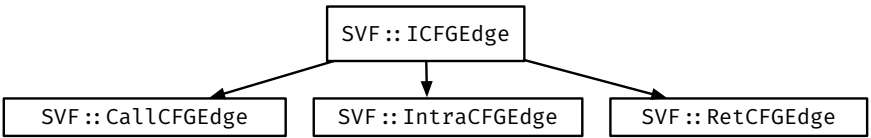
ICFG Node and Edge Classes



https://github.com/SVF-tools/SVF/blob/master/include/Graphs/ICFGNode.h

ICFG Node and Edge Classes





https://github.com/SVF-tools/SVF/blob/master/include/Graphs/ICFGEdge.h

`SVFUtil::cast` and `SVFUtil::dyn_cast`

- C++ Inheritance: see slides in Week 1 Lab.
- Casting a **parent** class pointer to pointer of a **Child** type:
  - `SVFUtil::cast`
    - Casts a pointer or reference to an instance of a specified class. This cast fails and aborts the program if the object or reference is not the specified class at runtime.
  - `SVFUtil::dyn_cast`
    - ”Checked cast” operation. Checks to see if the operand is of the specified type, and if so, returns a pointer to it (this operator does not work with references). If the operand is not of the correct type, a null pointer is returned.
    - Works very much like the dynamic_cast<> operator in C++, and should be used in the same circumstances.
- Example: accessing the attributes of the child class via casting.
  - `RetICFGNode* retNode = SVFUtil::cast<RetICFGNode>(ICFGNode);`
  - `CallCFGEdge* callEdge = SVFUtil::dyn_cast<CallCFGEdge>(ICFGEdge);`

“Casting” from Parent to Child in Python

- Python is a **dynamically typed language**:
  - No cast is required to call a method or access an attribute of an object.
  - A method can be called without declaring the object’s type in code, as long as the object is of the correct type at runtime.
- **Dynamic Typing $\neq$ No Discipline**:
  - **Adding type hints** is highly recommended to improve **readability** and **tooling support**.
  - This enables code completion, bug detection, and better maintainability.
- `isinstance()` and `typing.cast`:
  - Use `isinstance()` to narrow the type at runtime and assist IDE.
  - Use `typing.cast()` to explicitly annotate the expected type.
- Example: narrowing and annotating types for better IDE inference.
  - `if isinstance(node, RetICFGNode):` # IDE infers node is RetICFGNode
  - `call_edge = typing.cast(CallCFGEdge, edge)` # IDE infers call_edge is CallCFGEdge

Lab: Andersen's Points-To Analysis on Constraint Graph

(Week 2)

Yulei Sui, Mohamad Barbar, Hanyuan Li, Angela He

School of Computer Science and Engineering

University of New South Wales, Australia

- Constraint Graph
  - Modeling the relations between variables (e.g., pointers and memory allocations/objects)
  - Solving constraints is important to reason about program semantics (e.g., points-to relations)
- Points-to Analysis on Constraint Graph
  - What does the pointer *p* point to? Does *p* point to the same thing as *q*? Does *p* point to object *o*?
  - **And much more you may want to ask about pointers and what they will point to at runtime**
  - More about Andersen's analysis: `https://github.com/SVF-tools/Software-Security-Analysis/blob/slides/1.lab-andersen.pdf`
  - Implementation hints on the wiki: `https://github.com/SVF-tools/Software-Security-Analysis/wiki/Lab-Exercise-1#5-implementation-hints`
  - **This lab** covers Andersen's analysis on **self-constructed graphs**. **Week 3** will introduce constraint solving and Andersen **on LLVM and SVF IR**.

Scope

ADDR instructions: $p = \&o$

$$o \xrightarrow{Addr} p$$

Scope

ADDR instructions: $p = \&o$

$$o \xrightarrow{Addr} p$$

COPY instructions: $p = q$

$$q \xrightarrow{Copy} p$$

Scope

ADDR instructions: $p = \&o$

$$o \xrightarrow{Addr} p$$

STORE instructions: $*p = q$

$$q \xrightarrow{Store} p$$

COPY instructions: $p = q$

$$q \xrightarrow{Copy} p$$

Scope

ADDR instructions: $p = \&o$

$o \xrightarrow{Addr} p$

STORE instructions: $*p = q$

$q \xrightarrow{Store} p$

COPY instructions: $p = q$

$q \xrightarrow{Copy} p$

LOAD instructions: $p = *q$

$q \xrightarrow{Load} p$

Goal

A points-to set *pts* for every pointer and object.

For example, $pts(p) = \{o_1, o_2\}$, $pts(q) = \{o_1\}$, $pts(o_1) = \{\ \}$, ...

Constraint solving

- Our instructions form constraints
- We solve these constraints by manipulating points-to sets (all unions!)
- Keep “solving” these until we reach our goal
- We are done when we reach a fixpoint

Initial state

An empty points-to set for every pointer and every object.

Solving ADDR instructions

$$p = \&o$$

$$\hookrightarrow$$

$$pts(p) = pts(p) \cup \{o\}$$

Solving ADDR instructions

$p = \&o$

$\hookrightarrow$

$pts(p) = pts(p) \cup \{o\}$

$o \xrightarrow{ADDR} p$

$\hookrightarrow$

$o \xrightarrow{ADDR} p$

Solving COPY instructions

$$p = o$$

$$\hookrightarrow$$

$$pts(p) = pts(p) \cup pts(q)$$

Solving COPY instructions

$p = o$

$\hookrightarrow$

$pts(p) = pts(p) \cup pts(q)$

$q \xrightarrow{COPY} p$

$\hookrightarrow$

$q \xrightarrow{COPY} p$

Solving LOAD instructions

$$p = *q$$

$$\hookrightarrow$$

$$pts(p) = pts(p) \cup pts(*q)$$

Solving LOAD instructions

$p = *q$

$\hookrightarrow$

~~$pts(p) = pts(p) \cup pts(*q)$~~

$\forall o \in pts(q).\ pts(p) = pts(p) \cup pts(o)$

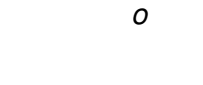
Solving LOAD instructions

$p = *q$

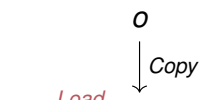
$\hookrightarrow$

~~$pts(p) = pts(p) \cup pts(*q)$~~

$\forall o \in pts(q).\ pts(p) = pts(p) \cup pts(o)$



$\hookrightarrow$ (assume $pts(q) = \{o\}$)



Solving STORE instructions

$$*p = q$$

$$\hookrightarrow$$

$$pts(*p) = pts(*p) \cup pts(q)$$

Solving STORE instructions

$$*p = q$$

$$\hookrightarrow$$

$$\cancel{pts(*p) = pts(*p) \cup pts(q)}$$

$$\forall o \in pts(p).\ pts(o) = pts(o) \cup pts(q)$$

Solving STORE instructions

$*p = q$

$\hookrightarrow$

~~$pts(*p) = pts(*p) \cup pts(q)$~~

$\forall o \in pts(p).\ pts(o) = pts(o) \cup pts(q)$

o

$q \xrightarrow{Store} p$

$\hookrightarrow$ (assume $pts(p) = \{o\}$)

o

Copy ↗

$q \xrightarrow{Store} p$

Algorithmically solving the constraints

We'll be using a worklist...

If a node's points-to set is ever changed, push it onto the worklist.

If a node is ever given a new outgoing COPY edge, push it onto the worklist.

Algorithmically solving the constraints

We'll be using a worklist...

If a node's points-to set is ever changed, push it onto the worklist.
If a node is ever given a new outgoing COPY edge, push it onto the worklist.

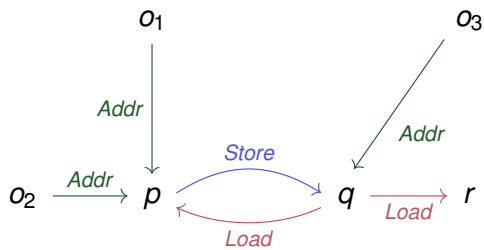
1. Initialise the graph with all the instructions.
2. Solve every ADDR instruction.
3. As long as the worklist is not empty...
   - 3.1 Pop node *n* from the worklist.
   - 3.2 Solve each incoming STORE edge and each outgoing LOAD edge of *n*.
   - 3.3 Solve each outgoing COPY edge of *n*.
4. We're done! We've reached a fixpoint!

Example – initialise the graph

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

$pt(p) = \{\}$ $pt(o_1) = \{\}$
$pt(q) = \{\}$ $pt(o_2) = \{\}$
$pt(r) = \{\}$ $pt(o_3) = \{\}$

Worklist: –

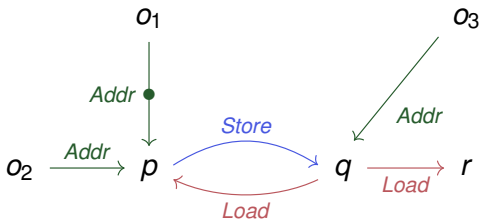


Example – solve ADDR instructions (1)

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1 ⟸
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: –

$pt(p) = \{\ \}$ $pt(o_1) = \{\ \}$
$pt(q) = \{\ \}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{\ \}$

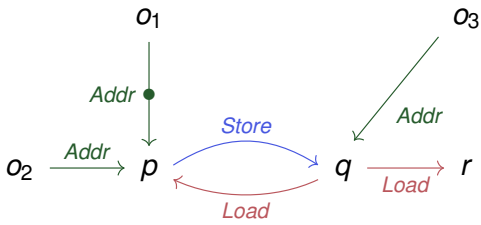


Example – solve ADDR instructions (1)

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1 ⟸
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: –

$pt(p) = \{o_1\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{\ \}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{\ \}$

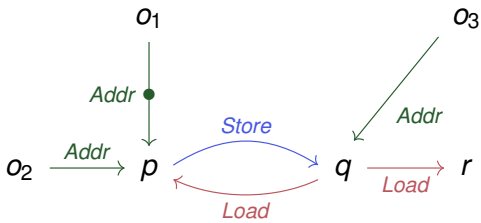


Example – solve ADDR instructions (1)

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1 ⟸
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: p

$pt(p) = \{o_1\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{\ \}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{\ \}$

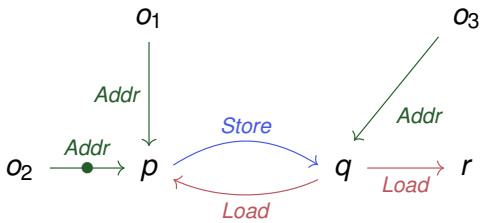


Example – solve ADDR instructions (2)

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2 ⟸
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: p

$pt(p) = \{o_1\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{\ \}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{\ \}$

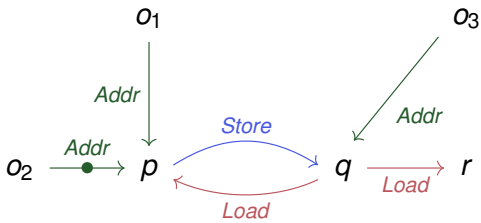


Example – solve ADDR instructions (2)

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2 ⟸
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: p

$pt(p) = \{o_1, \underline{o_2}\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{\ \}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{\ \}$

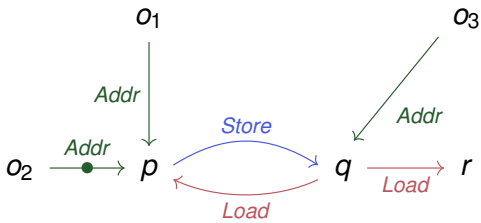


Example – solve ADDR instructions (2)

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2 ⟸
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: p, p

$pt(p) = \{o_1, \underline{o_2}\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{\ \}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{\ \}$

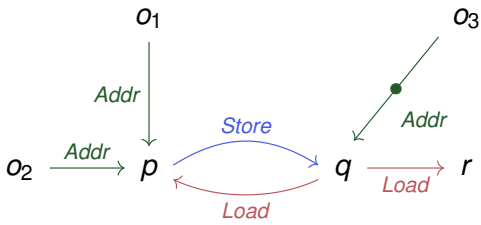


Example – solve ADDR instructions (3)

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3 ⟸
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: p, p

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{\ \}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{\ \}$

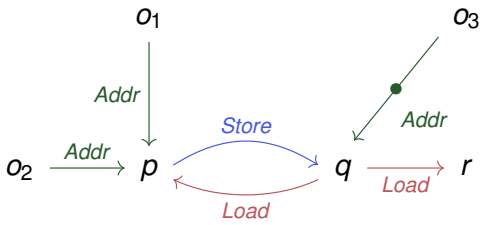


Example – solve ADDR instructions (3)

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3 ⟸
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: p, p

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{\ \}$

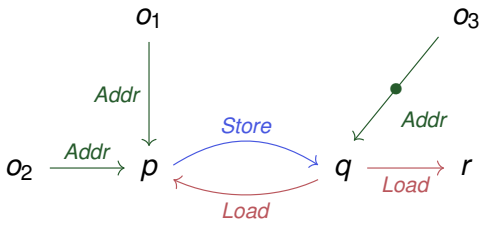


Example – solve ADDR instructions (3)

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3 ⟸
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: p, p, q

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{\ \}$

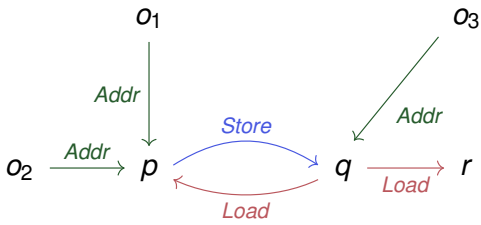


Example – worklist (1): p's incoming STOREs/outgoing LOADs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{p}$, p, q

$$pt(p) = \{o_1, o_2\} \qquad pt(o_1) = \{\,\}$$
$$pt(q) = \{o_3\} \qquad pt(o_2) = \{\,\}$$
$$pt(r) = \{\,\} \qquad pt(o_3) = \{\,\}$$

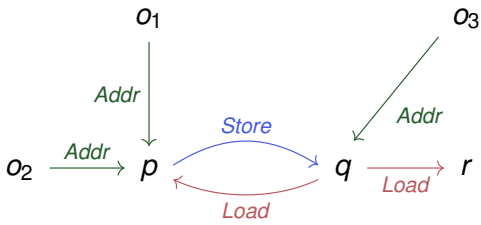


Example – worklist (1): p's outgoing COPYs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{p}$, p, q

$$pt(p) = \{o_1, o_2\} \qquad pt(o_1) = \{\ \}$$
$$pt(q) = \{o_3\} \qquad pt(o_2) = \{\ \}$$
$$pt(r) = \{\ \} \qquad pt(o_3) = \{\ \}$$

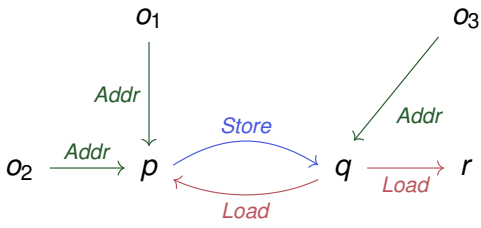


Example – worklist (2): p's incoming STOREs/outgoing LOADs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{p}$, q

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{\ \}$

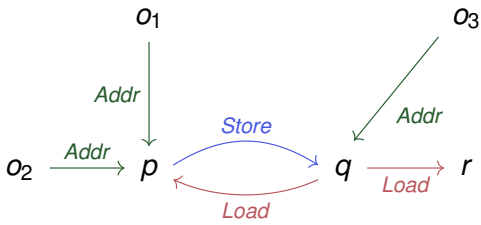


Example – worklist (2): p's outgoing COPYs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{p}$, q

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{\ \}$

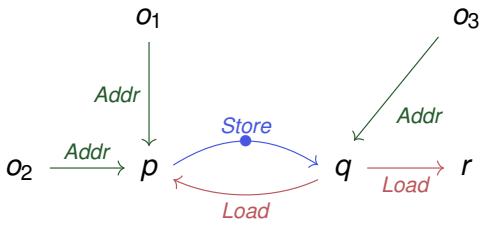


Example – worklist (3): q's incoming STOREs/outgoing LOADs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE ⟸
  p = *q;          // LOAD
}
```

Worklist: q

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{\ \}$

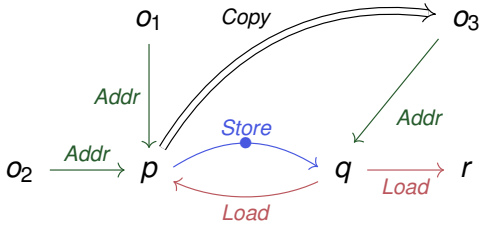


Example – worklist (3): q's incoming STOREs/outgoing LOADs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE ⟸
  p = *q;          // LOAD
}
```

Worklist: q

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{\ \}$

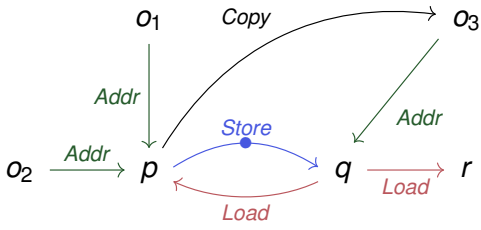


Example – worklist (3): q's incoming STOREs/outgoing LOADs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE ⟸
  p = *q;          // LOAD
}
```

Worklist: $\boxed{q}$, p

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{\ \}$

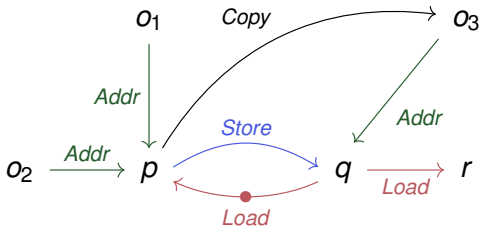


Example – worklist (3): q's incoming STOREs/outgoing LOADs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD ⟸
}
```

Worklist: $\boxed{q}$, p

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{\ \}$

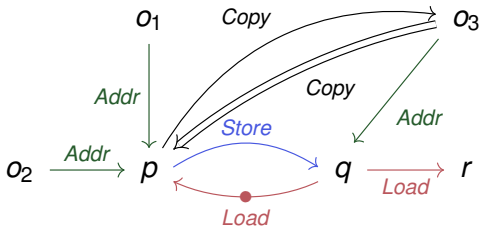


Example – worklist (3): q's incoming STOREs/outgoing LOADs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD ⟸
}
```

Worklist: $\boxed{q}$, p

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{\ \}$

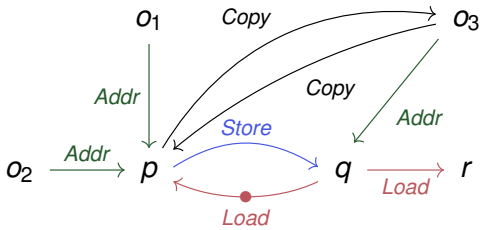


Example – worklist (3): q's incoming STOREs/outgoing LOADs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD ⟸
}
```

Worklist: $\boxed{q}$, p, o_3

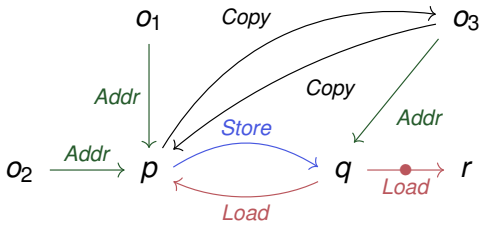
$$pt(p) = \{o_1, o_2\} \qquad pt(o_1) = \{\,\}$$
$$pt(q) = \{o_3\} \qquad pt(o_2) = \{\,\}$$
$$pt(r) = \{\,\} \qquad pt(o_3) = \{\,\}$$



Example – worklist (3): q's incoming STOREs/outgoing LOADs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD ⟸
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{q}$, p, o_3



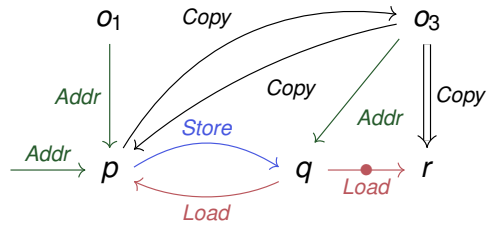
$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{\ \}$

Example – worklist (3): q's incoming STOREs/outgoing LOADs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD ⟸
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{q}$, p, o_3

$$pt(p) = \{o_1, o_2\} \qquad pt(o_1) = \{\ \}$$
$$pt(q) = \{o_3\} \qquad pt(o_2) = \{\ \}$$
$$pt(r) = \{\ \} \qquad pt(o_3) = \{\ \}$$

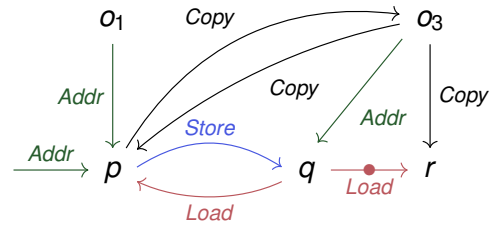


Example – worklist (3): q's incoming STOREs/outgoing LOADs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD ⟸
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{q}$, p, o_3, o_3

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{\ \}$

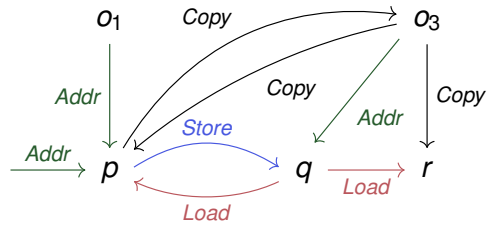


Example – worklist (3): q's outgoing COPYs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{q}$, p, o_3, o_3

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{\ \}$

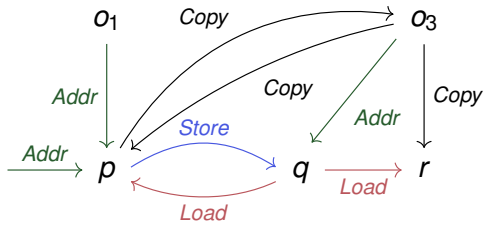


Example – worklist (4): p's incoming STOREs/outgoing LOADs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{p}$, o_3, o_3

$$pt(p) = \{o_1, o_2\} \qquad pt(o_1) = \{\ \}$$
$$pt(q) = \{o_3\} \qquad pt(o_2) = \{\ \}$$
$$pt(r) = \{\ \} \qquad pt(o_3) = \{\ \}$$

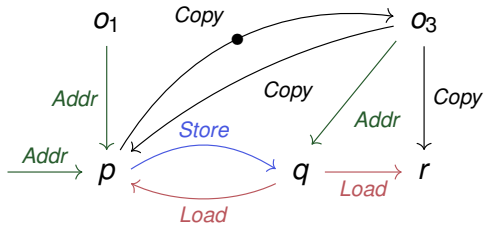


Example – worklist (4): p's outgoing COPYs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{p}$, o_3, o_3

$$pt(p) = \{o_1, o_2\} \qquad pt(o_1) = \{\,\} \qquad pt(o_2) = \{\,\}$$
$$pt(q) = \{o_3\} \qquad pt(o_2) = \{\,\} \qquad pt(o_3) = \{\,\}$$
$$pt(r) = \{\,\}$$

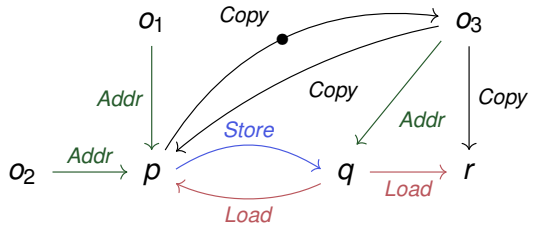


Example – worklist (4): p's outgoing COPYs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{p}$, o_3, o_3

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{o_1, o_2\}$

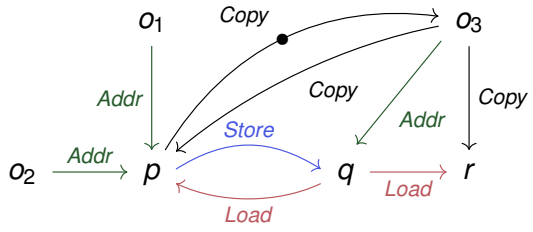


Example – worklist (4): p's outgoing COPYs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{p}$, o_3, o_3, o_3

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{o_1, o_2\}$

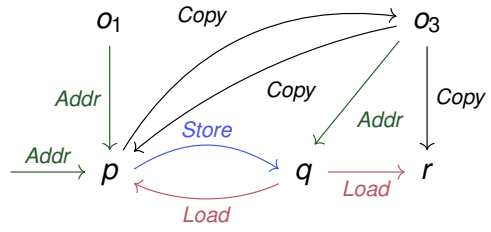


Example – worklist (5): o_3's incoming STOREs/outgoing LOADs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{o_3}$, o_3, o_3

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{o_1, o_2\}$

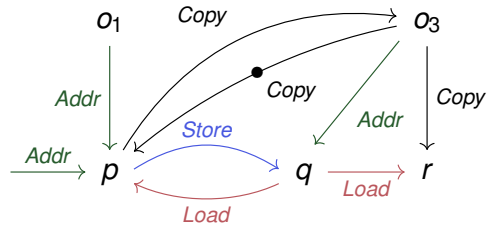


Example – worklist (5): o_3's outgoing COPYs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{o_3}$, o_3, o_3

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{o_1, o_2\}$

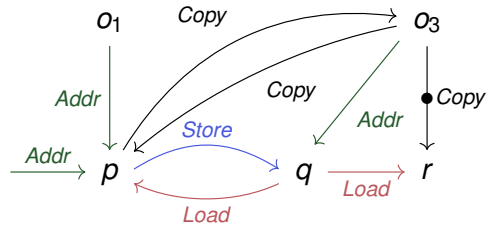


Example – worklist (5): o_3's outgoing COPYs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{o_3}$, o_3, o_3

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{\ \}$ $pt(o_3) = \{o_1, o_2\}$

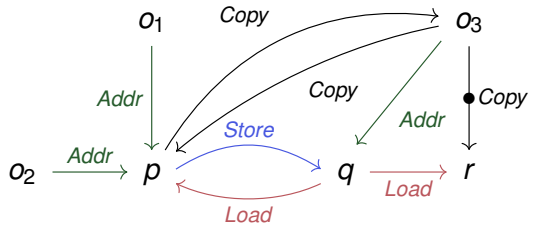


Example – worklist (5): o_3's outgoing COPYs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{o_3}$, o_3, o_3

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{o_1, o_2\}$ $pt(o_3) = \{o_1, o_2\}$

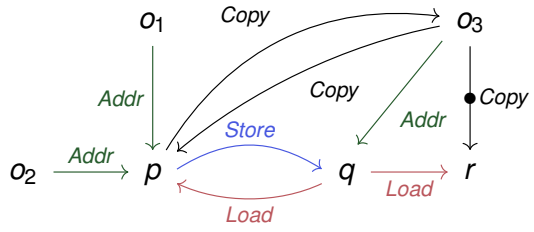


Example – worklist (5): o_3's outgoing COPYs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{o_3}$, o_3, o_3, r

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{o_1, o_2\}$ $pt(o_3) = \{o_1, o_2\}$

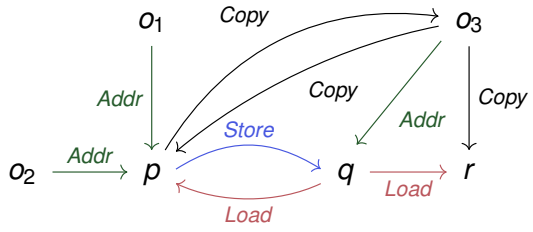


Example – worklist (6): o_3's incoming STOREs/outgoing LOADs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{o_3}$, o_3, r

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{o_1, o_2\}$ $pt(o_3) = \{o_1, o_2\}$

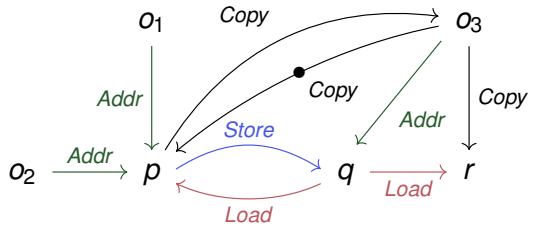


Example – worklist (6): o_3's outgoing COPYs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{o_3}$, o_3, r

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{o_1, o_2\}$ $pt(o_3) = \{o_1, o_2\}$

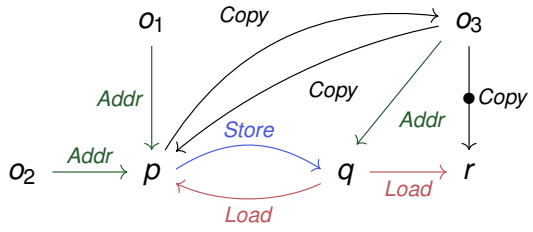


Example – worklist (6): o_3's outgoing COPYs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{o_3}$, o_3, r

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{o_1, o_2\}$ $pt(o_3) = \{o_1, o_2\}$

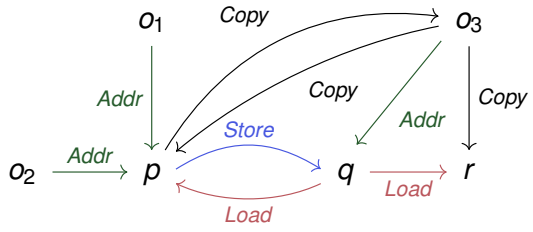


Example – worklist (7): o_3's incoming STOREs/outgoing LOADs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{o_3}$, r

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{o_1, o_2\}$ $pt(o_3) = \{o_1, o_2\}$

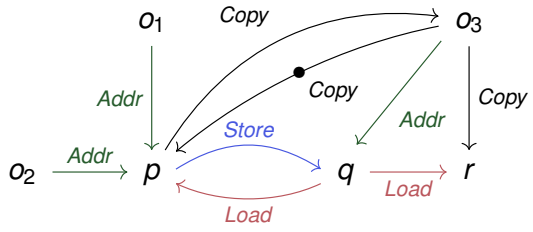


Example – worklist (7): o_3's outgoing COPYs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{o_3}$, r

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{o_1, o_2\}$ $pt(o_3) = \{o_1, o_2\}$

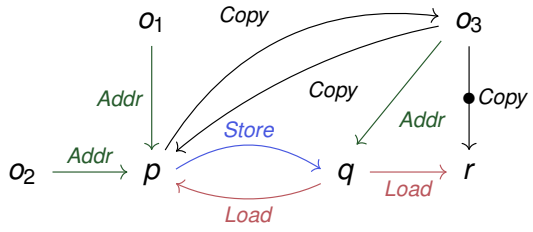


Example – worklist (7): o_3's outgoing COPYs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: $\boxed{o_3}$, r

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{o_1, o_2\}$ $pt(o_3) = \{o_1, o_2\}$

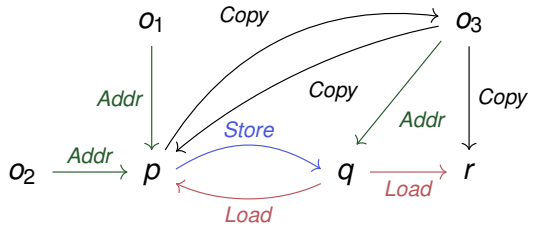


Example – worklist (8): r's incoming STOREs/outgoing LOADs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: r

$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{o_1, o_2\}$ $pt(o_3) = \{o_1, o_2\}$

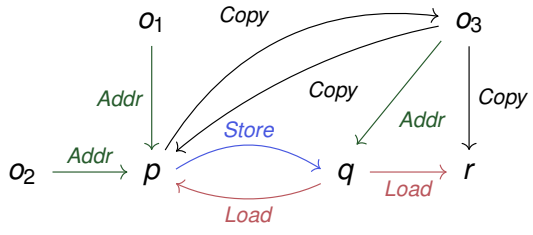


Example – worklist (8): *r*'s outgoing COPYs

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

Worklist: r

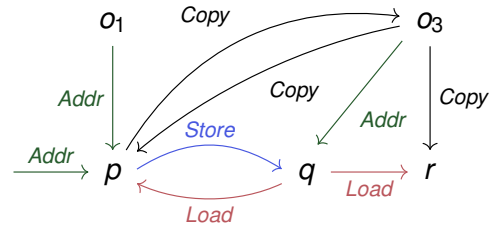
$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{o_1, o_2\}$ $pt(o_3) = \{o_1, o_2\}$



Example – fixpoint!

```
int main(void) {
  int *p, **q, *r;
  p = malloc(...); // ADDR o1
  p = malloc(...); // ADDR o2
  q = malloc(...); // ADDR o3
  r = *q;          // LOAD
  *q = p;          // STORE
  p = *q;          // LOAD
}
```

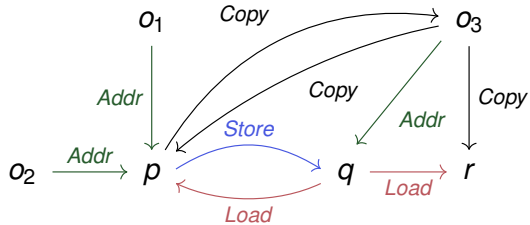
Worklist: –



$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{o_1, o_2\}$ $pt(o_3) = \{o_1, o_2\}$

Example – tainted flows

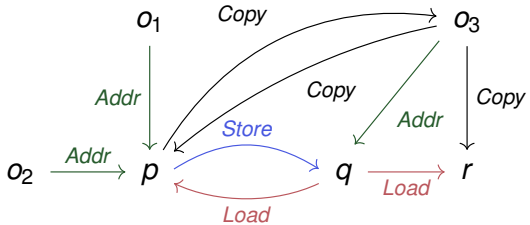
```
int main(void) {
  int *p, **q, *r;
  p = sensitive(...); // ADDR o1
  p = malloc(...);    // ADDR o2
  q = malloc(...);    // ADDR o3
  r = *q;             // LOAD
  *q = p;             // STORE
  p = *q;             // LOAD
  broadcast(p);  // Safe?
  broadcast(q);  // Safe?
  broadcast(r);  // Safe?
}
```



$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{o_1, o_2\}$ $pt(o_3) = \{o_1, o_2\}$

Example – tainted flows

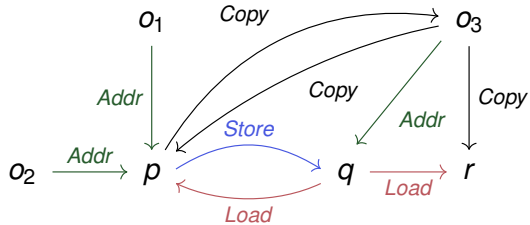
```
int main(void) {
  int *p, **q, *r;
  p = sensitive(...); // ADDR o1
  p = malloc(...);    // ADDR o2
  q = malloc(...);    // ADDR o3
  r = *q;             // LOAD
  *q = p;             // STORE
  p = *q;             // LOAD
  broadcast(p);  // Safe?  NO
  broadcast(q);  // Safe?
  broadcast(r);  // Safe?
}
```



$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{o_1, o_2\}$ $pt(o_3) = \{o_1, o_2\}$

Example – tainted flows

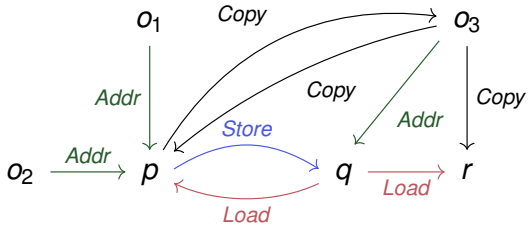
```
int main(void) {
  int *p, **q, *r;
  p = sensitive(...); // ADDR o1
  p = malloc(...);    // ADDR o2
  q = malloc(...);    // ADDR o3
  r = *q;             // LOAD
  *q = p;             // STORE
  p = *q;             // LOAD
  broadcast(p);  // Safe?  NO
  broadcast(q);  // Safe?  YES
  broadcast(r);  // Safe?
}
```



$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{o_1, o_2\}$ $pt(o_3) = \{o_1, o_2\}$

Example – tainted flows

```
int main(void) {
  int *p, **q, *r;
  p = sensitive(...); // ADDR o1
  p = malloc(...);    // ADDR o2
  q = malloc(...);    // ADDR o3
  r = *q;             // LOAD
  *q = p;             // STORE
  p = *q;             // LOAD
  broadcast(p);  // Safe?  NO
  broadcast(q);  // Safe?  YES
  broadcast(r);  // Safe?  NO
}
```



$pt(p) = \{o_1, o_2\}$ $pt(o_1) = \{\ \}$
$pt(q) = \{o_3\}$ $pt(o_2) = \{\ \}$
$pt(r) = \{o_1, o_2\}$ $pt(o_3) = \{o_1, o_2\}$